

```
import re
import numpy as np
import pandas as pd
```

```
ARQUIVO = r"Avaliação de Nível de Maturidade de Engenharia Software Verde (respostas)
(1) (1).xlsx"
```

```
ABA = None # opcional: "Respostas ao formulário 1"
```

```
ITEM_COL_RE = re.compile(r"^s*\d+\s*-")
```

```
def carregar_aba(path: str, aba: str | None) -> tuple[pd.DataFrame, str]:
```

```
    xls = pd.ExcelFile(path)
```

```
    if aba is not None:
```

```
        return pd.read_excel(path, sheet_name=aba), aba
```

```
    # escolhe automaticamente a aba com mais colunas de itens
```

```
    best_df, best_sh, best_n = None, None, -1
```

```
    for sh in xls.sheet_names:
```

```
        df = pd.read_excel(path, sheet_name=sh)
```

```
        n = sum(1 for c in df.columns if ITEM_COL_RE.match(str(c)))
```

```
        if n > best_n:
```

```
            best_df, best_sh, best_n = df, sh, n
```

```
    return best_df, best_sh
```

```
def parse_cell(x):
```

```
    """Converte célula do tipo '3 - Parcialmente' -> 3. Caso falhe, NaN."""
```

```
    if pd.isna(x):
```

```
        return np.nan
```

```
    if isinstance(x, (int, float, np.integer, np.floating)):
```

```
        return float(x)
```

```
    s = str(x).strip()
```

```
    m = re.match(r"^s*(\d+)", s)
```

```
    return float(m.group(1)) if m else np.nan
```

```
def extrair_itens(df: pd.DataFrame) -> pd.DataFrame:
```

```
    item_cols = [c for c in df.columns if ITEM_COL_RE.match(str(c))]
```

```
    if not item_cols:
```

```
        raise ValueError("Não encontrei colunas de itens no padrão '1 - ...'.")
```

```
    itens_num = df[item_cols].apply(lambda col: col.map(parse_cell))
```

```
    # Casos completos (como descrito no texto)
```

```
    itens_num = itens_num.dropna(axis=0, how="any")
```

```
    return itens_num
```

```

def cronbach_alpha_classico(itens_num: pd.DataFrame) -> dict:
    """
     $\alpha = k/(k-1) * (1 - (\sum s_i^2 / s_T^2))$ 
     $\bar{r}$  = média das correlações inter-itens (padrão, sem “forçar” por  $\alpha$  arredondado)
    """

    N, k = itens_num.shape
    if N < 2 or k < 2:
        raise ValueError("Precisa de N>=2 e k>=2.")

    variancias_itens = itens_num.var(axis=0, ddof=1)
    sum_s2 = float(variancias_itens.sum())

    escore_total = itens_num.sum(axis=1)
    s2_T = float(escore_total.var(ddof=1))
    if s2_T == 0:
        raise ValueError("Variância total zero; alfa indefinido.")

    alpha = (k / (k - 1)) * (1 - (sum_s2 / s2_T))

    #  $\bar{r}$  padrão (média das correlações fora da diagonal)
    corr = itens_num.corr()
    rbar = float(corr.values[np.triu_indices(k, 1)].mean())

    return {
        "N": int(N),
        "k": int(k),
        "sum_s2_itens": sum_s2,
        "s2_total": s2_T,
        "alpha": float(alpha),
        "rbar": rbar,
        "ratio": float(sum_s2 / s2_T),
    }

def main():
    df, aba_usada = carregar_aba(ARQUIVO, ABA)
    itens = extrair_itens(df)
    res = cronbach_alpha_classico(itens)

    print(f"[INFO] Aba usada: {aba_usada}")
    print("\n=== ITENS INDIVIDUAIS (35 itens) — CÁLCULO CLÁSSICO ===")
    print(f"N (casos válidos): {res['N']}")
    print(f"k: {res['k']}")
    print(f" $\sum s_i^2$  (soma variâncias): {res['sum_s2_itens']:.6f}")
    print(f" $s_T^2$  (variância do escore total): {res['s2_total']:.6f}")
    print(f" $\alpha$  (Cronbach): {res['alpha']:.6f}")
    print(f" $\bar{r}$  (média inter-itens): {res['rbar']:.6f}")

```

```
print(f" $\sum s^2_i / s^2_T$ : {res['ratio']:.6f}")
```

```
if __name__ == "__main__":  
    main()
```